

RELATÓRIO DE TESTES AUTOMATIZADOS: API SERVEREST

Período: Sprint 2 - Semana 4 | Desafio 3

Desenvolvedora de Automação de Testes Trainee: Letícia Vidal

1. RESUMO DO PLANEJAMENTO DE TESTES

Este documento apresenta os resultados da automação de testes realizada na API ServeRest. A estratégia focou na transição do planejamento exploratório manual para uma suíte automatizada focada em estabilidade, cobrindo fluxos críticos de Usuários, Login, Produtos e Carrinhos.

1.1 Escopo da Automação

- **Endpoints Validados:** /usuarios , /login , /produtos , /carrinhos .
- **Abordagem:** Testes de caminho feliz (positivos), validações de erro (negativos) e testes de segurança/regressão.
- **Massa de Dados:** Implementação de geração de massa dinâmica para e-mails e nomes reais, levando em consideração a simulação de um ecommerce na API e a maior variedade de dados, o que possibilita mais testes realistas.

1.2 Evolução: Manual (Sprint-6 [Fase-1]) VS. Automação (Sprint-2 [Fase-2])

Durante a fase de testes manuais (Sprint-6), foram mapeados 35 cenários focados em exploração ampla. Para a etapa de automação, utilizei a técnica de Particionamento de Equivalência para otimizar o escopo, resultando na seleção técnica de **17 cenários principais**. O foco da automação deixou de ser a exploração exaustiva para se concentrar na validação estrita de contratos, regras de negócio e prevenção de regressão em falhas de segurança.

2. LISTA DE CENÁRIOS EXECUTADOS

Abaixo, os 17 cenários automatizados e validados nos scripts, categorizados por sua finalidade técnica:

Módulo: Login

- **Cenário 01 [POSITIVO]:** Login realizado com sucesso (Geração de Token Bearer).
- **Cenário 02 [NEGATIVO]:** Login com senha inválida (Bloqueio 401).
- **Cenário 03 [NEGATIVO]:** Login com e-mail não cadastrado (Bloqueio 401).
- **Cenário 04 [SEGURANÇA]:** Acesso com Token Malformado (Validação de integridade do Header).

Módulo: Usuários

- **Cenário 01 [POSITIVO]:** Cadastro de usuário comum com sucesso.
- **Cenário 02 [REGRA DE NEGÓCIO]:** Impedir cadastro de e-mail duplicado (Regra de Unicidade).
- **Cenário 03 [SEGURANÇA / REGRESSÃO]:** Validar IDOR na exclusão (Teste de Permissão entre usuários).

Módulo: Produtos

- **Cenário 01 [POSITIVO]:** Cadastro de produto com sucesso (Perfil Admin).
- **Cenário 02 [REGRA DE NEGÓCIO]:** Bloquear cadastro de produto por usuário comum (RBAC - Controle de Acesso).
- **Cenário 03 [REGRA DE NEGÓCIO]:** Impedir cadastro de produto com nome já existente.
- **Cenário 04 [NEGATIVO]:** Validar erro ao cadastrar produto com nome vazio.
- **Cenário 05 [NEGATIVO]:** Cadastrar produto com preço negativo.
- **Cenário 06 [NEGATIVO]:** Cadastrar produto com descrição excessiva (5000 caracteres / Análise de Valor Limite).

Módulo: Carrinhos

- **Cenário 01 [POSITIVO]:** Criar carrinho com sucesso.
- **Cenário 02 [REGRA DE NEGÓCIO]:** Impedir mais de um carrinho por usuário.
- **Cenário 03 [REGRA DE NEGÓCIO]:** Validar erro de estoque insuficiente.
- **Cenário 04 [NEGATIVO / REGRESSÃO]:** Status Code ao concluir compra inexistente.

3. EVIDÊNCIAS DE EXECUÇÃO

3.1 Resumo Geral (Terminal) A execução foi realizada via CLI, resultando em:

- Total de Testes: 17
- Sucesso (PASS): 14
- Falhas (FAIL): 3 (Bugs de Regressão/Segurança)

Nota: Os logs detalhados e o relatório visual interativo são gerados na pasta `./results/log.html`, contendo o passo a passo de cada requisição HTTP enviada e recebida.

Details

all

Tags

Results

Search

Status:

17 tests total, 14 passed, 3 failed, 0 skipped

Total Time:

00:00:08.853

Test Name	Documentation	Tags	Status	Message	Elapsed	Start	End
Test Case: Cenário de Regressão: Concluir compra sem carrinho			FAIL	BUG DETECTADO: Concluir sem carrinho. (Status: 200)	00:00:00.397	20260319 21:57:03.642	20260319 21:57:03.638
Test Case: Cenário de Segurança: Acesso com Token Malformado			FAIL	BUG DETECTADO: API não trata token malformado. (Status: 200)	00:00:00.199	20260319 21:57:03.315	20260319 21:57:03.312
Test Case: Cenário de Regressão: Validar IDOR na exclusão			FAIL	BUG DETECTADO: Falha de IDOR detectada. (Status: 200)	00:00:00.796	20260319 21:57:07.681	20260319 21:57:07.839
Test Case: Cenário 01: Citar carrinho com sucesso			PASS		00:00:01.275	20260319 21:56:58.502	20260319 21:57:00.209
Test Case: Cenário 02: Impedir mais de um carrinho			PASS		00:00:00.815	20260319 21:57:00.207	20260319 21:57:01.022
Test Case: Cenário de Regressão: Estoque insuficiente			PASS		00:00:00.618	20260319 21:57:01.024	20260319 21:57:01.642
Test Case: Cenário 01: Login realizado com sucesso			PASS		00:00:00.621	20260319 21:57:02.057	20260319 21:57:02.678
Test Case: Cenário 02: Login com senha inválida			PASS		00:00:00.428	20260319 21:57:02.678	20260319 21:57:03.107
Test Case: Cenário 02: Login com e-mail não cadastrado			PASS		00:00:00.208	20260319 21:57:03.106	20260319 21:57:03.311
Test Case: Cenário 01: Cadastrar produto com sucesso como Admin (CT16)		admin, produto	PASS		00:00:00.824	20260319 21:57:03.303	20260319 21:57:04.349
Test Case: Cenário 02: Recuperar cadastro de produto por usuário comum (CT19)		negativo, permissao	PASS		00:00:00.102	20260319 21:57:04.349	20260319 21:57:04.544
Test Case: Cenário 03: Impedir cadastro de produto com nome já existente (CT20)		negativo, regra_negocio	PASS		00:00:00.418	20260319 21:57:04.544	20260319 21:57:05.365
Test Case: Cenário de Limite: Cadastrar produto com descrição excessiva		estresse, negativo	PASS		00:00:00.229	20260319 21:57:05.365	20260319 21:57:05.782
Test Case: Cenário de Regressão: Cadastrar produto com preço negativo		integridade, regressao	PASS		00:00:00.211	20260319 21:57:05.569	20260319 21:57:05.780
Test Case: Cenário de Regressão: Validar erro ao cadastrar produto com nome vazio		regressao, validacao	PASS		00:00:00.216	20260319 21:57:05.780	20260319 21:57:05.568
Test Case: Cenário 01: Cadastrar de usuário comum com sucesso			PASS		00:00:00.627	20260319 21:57:05.568	20260319 21:57:06.650
Test Case: Cenário 02: Impedir e-mail duplicado			PASS		00:00:00.390	20260319 21:57:06.650	20260319 21:57:07.041

Tests Report										Generated 20260319 21:57:07 UTC-03:00 15 minutes 40 seconds ago	
Summary											
Status:	3 tests failed										
Start Time:	20260319 21:56:58.003										
End Time:	20260319 21:57:07.844										
Elapsed Time:	00:00:09.841										
Log File:	log.html										
Statistics											
Total Statistics											
	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip					
All Tests	17	14	3	0	00:00:09						
Statistics by Tag											
	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip					
admin	1	1	0	0	00:00:01						
estresse	1	1	0	0	00:00:00						
integridade	1	1	0	0	00:00:00						
negativo	3	3	0	0	00:00:01						
permissao	1	1	0	0	00:00:01						
positivo	1	1	0	0	00:00:01						
regra_negocio	1	1	0	0	00:00:00						
regressao	2	2	0	0	00:00:00						
validacao	1	1	0	0	00:00:00						
Statistics by Suite											
	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip					
Tests	17	14	3	0	00:00:10						
Suite: Carrinho	4	3	1	0	00:00:04						
Suite: Login	4	3	1	0	00:00:01						
Suite: Produtos	6	6	0	0	00:00:02						
Suite: Usuarios	3	2	1	0	00:00:02						

4. LISTA DE BUGS ENCONTRADOS (ISSUES)

Durante a execução da suíte, foram detectados 3 bugs onde o comportamento da API diverge do esperado:

- **BUG-01: Vulnerabilidade de Segurança - Token Malformado**
 - **Comportamento:** A API aceita tokens de autorização inválidos ou corrompidos.
 - **Resultado:** Recebido 200 OK quando o esperado era 401 Unauthorized.
- **BUG-02: Vulnerabilidade Crítica - IDOR na Exclusão**

- **Comportamento:** Um usuário comum (não-admin) consegue excluir o registro de outro usuário através do ID.
- **Resultado:** Recebido 200 OK quando o esperado era 403 Forbidden.
- **BUG-03: Erro de Lógica - Concluir Compra Inexistente**
 - **Comportamento:** O sistema permite concluir a compra mesmo quando não há carrinho ativo para o usuário.
 - **Resultado:** Recebido 200 OK quando o esperado era 400 Bad Request.

< Voltar

ECS-9 / ECS-10

+

🔍

▼ Descrição

Título: [Login/Geral] API permite consumo de endpoints restritos utilizando Bearer Token inválido ou corrompido.

Descrição:

Durante a execução da suíte de testes de segurança, identificou-se que a API ServeRest não realiza a validação de integridade ou assinatura dos tokens enviados no header `Authorization`. Ao enviar um token malformado (ex: caracteres aleatórios, token incompleto ou modificado), a API ignora a invalidade do hash e processa a requisição como se o usuário estivesse devidamente autenticado.

Impacto:

A quebra do mecanismo de autenticação é uma falha crítica de segurança. Isso permite que:

- Usuários não autenticados burliem a camada de segurança da API.
- Ações restritas (como gestão de produtos e carrinhos) sejam executadas sem uma sessão válida.
- O sistema fique exposto a ataques de personificação e bypass de login.

Ambiente de Teste:

- **Sistema Operacional:** Windows 11.
- **Ferramenta de Execução:** Robot Framework 7.0 (RequestsLibrary).
- **API:** ServeRest.
- **Massa de Dados:** Strings aleatórias geradas para simular o token corrompido.

Pré-condições:

1. Possuir o endpoint de uma rota restrita (ex: `/produtos` ou `/carrinhos`).
2. Definir uma string arbitrária e malformada para ser utilizada como token (ex: `abc123corrompido`).

Passo a Passo para Reproduzir (via Script de Automação):

1. Configurar um cabeçalho de requisição com o campo `Authorization: Bearer abc123corrompido`.
2. Realizar uma requisição `POST` ou `SET` para um endpoint que exija autenticação.
3. Validar o retorno de Status Code e o corpo da resposta da API.

Comportamento Esperado: A API deve validar a estrutura do token e, ao detectar que ele é inválido/malformado, retornar **Status Code 401 Unauthorized** com a mensagem "Token ausente, inválido ou expirado".

Comportamento Atual:

A API ignora a malformação do token, aceita a requisição e retorna **Status Code 200 OK** (ou 201), permitindo o acesso indevido ao recurso.

Evidências:

- **Log do Robot Framework:** O cenário com a tag `seguranca` ou `integridade` apresenta falha na asserção (Esperado: 401 | Obtido: 200).
- **Relatório Visual:** Falha contabilizada na suíte de `Tests.Login`.

Frequência: 100% das execuções.

Gravidade (Severity): Alta (Vulnerabilidade de Autenticação).

Prioridade (Priority): P1 - Urgente.

< Voltar

SC9-9 / SC9-8

+

🔒

✓ Descrição

Título: API permite que um usuário comum exclua outro usuário sem validação de permissão (IDOR).

Descrição: Ao realizar a automação do fluxo de exclusão, identifiquei-se que a API não valida a propriedade do recurso ou o nível de privilégio. Um usuário "A" autenticado consegue excluir o registro de um usuário "B" apenas possuindo o seu `_id`. A API não verifica se o portador do Bearer Token é o dono da conta ou um Administrador, configurando uma falha grave de **Insecure Direct Object Reference (IDOR)**.

Impacto:

A segurança e a integridade da base de dados ficam comprometidas. Qualquer usuário autenticado pode deletar contas de terceiros em massa, resultando em:

- Exclusão indevida de contas e perda de dados.
- Quebra total do controle de acesso (RBAC).
- Comprometimento da confiabilidade da aplicação para uso em produção.

Ambiente de Teste:

- Sistema Operacional:** Windows 11.
- Ferramenta de Execução:** Robot Framework 7.0 (RequestsLibrary).
- API:** Serverless Local/Cloud.
- Massa de Dados:** Dinâmica (gerada via FakerLibrary).

Pré-condições:

- Existência de dois usuários comuns cadastrados na base (Usuário A e Usuário B).
- Possuir um Bearer Token válido para o Usuário A.
- Possuir o `_id` gerado para o Usuário B.

Passo a Passo para Reproduzir (via Script de Automação):

- Executar a Keyword de criação de usuário para gerar o **Usuário B** e armazenar seu `_id`.
- Executar a Keyword de Login com as credenciais do **Usuário A** para obter seu `token`.
- Realizar uma requisição **DELETE** para o endpoint `/usuarios/{id_do_usuario_B}` enviando o `token` do **Usuário A** no header de autorização.
- Validar o retorno da API.

Comportamento Esperado: A API deve retornar **Status Code 403 Forbidden** (ou 401, dependendo da regra), impedindo a exclusão e informando que o usuário não possui permissão para este recurso.

Comportamento Atual: A API processa a exclusão com sucesso, retornando **Status Code 200 OK** e a mensagem "Registro excluído com sucesso".

Evidências:

- Log do Robot Framework:** O teste marcado com a tag `[[excluir]]` apresenta falha na asserção de Status Code (Esperado: 403 | Obtido: 200).
- JSON de Resposta:** Demonstra o sucesso da operação indevida.

Frequência: 100% das execuções.

Gravidade (Severity): Alta (Falha de Segurança).

Prioridade (Priority): P2

< Voltar

SC9-9 / SC9-8

+

🔒

✓ Descrição

Título: [Carrinhos] API permite a conclusão de compra (checkout) para usuários sem carrinho ativo ou com carrinho vazio.

Descrição: Identifiquei-se uma falha na regra de negócio do endpoint de finalização de compras. A API permite que a requisição de conclusão de compra seja processada com sucesso mesmo que o usuário autenticado não possua um carrinho cadastrado no sistema. Não há uma validação prévia da existência do recurso "carrinho" antes de executar a ação de checkout.

Impacto: Este erro de lógica compromete a integridade do fluxo comercial da aplicação, podendo resultar em:

- Inconsistência de dados nos relatórios de vendas e pedidos.
- Processamento de transações fantasmas (sem itens ou valores).
- Experiência confusa para o usuário final, que recebe uma confirmação de sucesso para uma operação que não continha produtos.

Ambiente de Teste:

- Sistema Operacional:** Windows 11.
- Ferramenta de Execução:** Robot Framework 7.0 (RequestsLibrary).
- API:** Serverless.
- Massa de Dados:** Usuário recém-criado (sem histórico de carrinho).

Pré-condições:

- Existência de um usuário comum cadastrado.
- O usuário **não** deve possuir um carrinho ativo (ou o carrinho deve ter sido excluído previamente).
- Possuir o Bearer Token válido deste usuário.

Passo a Passo para Reproduzir (via Script de Automação):

- Realizar o login com um usuário que não possui carrinho para obter o `token`.
- Enviar uma requisição **DELETE** (ou o método correspondente à finalização) para o endpoint `/carrinhos/conclui-compra`.
- Validar o retorno de Status Code e a mensagem da API.

Comportamento Esperado: A API deve validar a existência do carrinho e, caso não exista, retornar **Status Code 400 Bad Request** com uma mensagem explicativa (ex: "Não foi encontrado carrinho para este usuário").

Comportamento Atual: A API aceita a finalização da compra inexistente, retornando **Status Code 200 OK** e a mensagem "Registro excluído com sucesso".

Evidências:

- Log do Robot Framework:** O cenário na suite `Tests.Carrinho` apresenta falha de asserção (Esperado: 400 | Obtido: 200).
- Relatório Visual:** Uma falha contabilizada no módulo de `Carrinho`.

Frequência: 100% das execuções. **Gravidade (Severity):** Média (Erro de Regra de Negócio). **Prioridade (Priority):** P2 - Média.

5. EXPLICAÇÃO DOS CHECKS AUTOMATIZADOS

Os testes foram estruturados seguindo o padrão de camadas (Resources, Data Factory, Tests):

- Status Codes:** Verificação rigorosa dos códigos HTTP retornados pela API ServeRest.
- Integridade de Dados:** Garantia de que campos obrigatórios de contrato estão presentes na resposta JSON.

- **Regras de Negócio:** Testes focados em cálculos de estoque, unicidade de e-mail e permissões baseadas em perfil (Admin vs Comum).
 - **Segurança:** Verificação de manipulação de headers e bypass de controle de acesso.
 - **Massa Dinâmica:** O uso da FakerLibrary evita falsos-positivos causados por dados de testes antigos (hardcoded) ou conflitos no banco de dados da aplicação.
-

6. INSTRUÇÕES DE EXECUÇÃO

Para reproduzir os testes apresentados neste relatório:

1. Clone o repositório e acesse a pasta:

```
1 | Bash
1 | cd Semana-4
```

2. Instale as dependências:

```
1 | Bash
1 | pip install robotframework robotframework-requests robotframework-faker
```

3. Execute a suíte:

```
1 | Bash
1 | robot -d ./results tests/
```

4. Visualize o Log: Abra o arquivo `results/log.html` no seu navegador.